

Quantization for Efficient LLM Inference: A Quality–Memory–Speed Study of Qwen2.5-7B-Instruct

Hubert Michalski Przemysław Fuchs

University of Warsaw {hm438596, pf438429}@students.mimuw.edu.pl

Abstract

Quantization lets large language models run on cheaper hardware, but a smaller model is not automatically a better one: it can lose accuracy, run slower in some runtimes, or save disk without saving the memory that actually limits inference. We measure these trade-offs for one 7B-class instruction model, Qwen2.5-7B-Instruct. Against a bf16 baseline we compare 8-bit and 4-bit post-training quantization (LLM.int8(), NF4, and official GPTQ and AWQ) on capability benchmarks (MMLU, GSM8K, HumanEval, IFEval) and a set of efficiency measurements on a single GPU. The 4-bit GPTQ and AWQ checkpoints give the best overall trade-off: they are the only variants that cut both disk and GPU memory while staying close to baseline speed, at a modest quality cost. Two results run against intuition: LLM.int8() is the slowest variant of all and saves no disk, and NF4 saves memory but still decodes slower than bf16. Together they show that quantization is mainly a memory technique rather than a speed one. We also implement and run four catalogued extensions on the same box: a JudgeBench meta-evaluation, W8A8/SmoothQuant self-quantization, a quantized KV cache, and QLoRA recovery. These sharpen the memory-versus-speed picture.

1 Introduction and Research Questions

Many LLMs are too expensive to serve in full precision, and post-training quantization is the standard remedy. Several practical methods exist: LLM.int8() [3] for 8-bit weights with mixed-precision handling of outliers; GPTQ [5] and AWQ [10], which use a small calibration set to push weights to 4 bits; SmoothQuant [15] for 8-bit weights and activations; the NF4 data type from QLoRA [4] for 4-bit storage in the bitsandbytes ecosystem; and KV-cache quantization such as KIVI [11] for long-context inference. None of these is automatically beneficial: the same 4-bit model can be smaller yet slower, or degrade unevenly across capabilities, so the practical trade-off is what we set out to measure. We organise the study around four questions from our proposal:

RQ1 Which method gives the best quality–memory–speed trade-off for a 7B-class instruction LLM?

RQ2 Are some capabilities (reasoning, coding, QA, instruction following) more sensitive to quantization than others?

RQ3 Does quantization improve real inference speed, or mainly reduce memory?

RQ4 (optional) Can light QLoRA-style adaptation recover quality lost after 4-bit quantization?

Our core study is the proposal’s minimal successful version (one model, five variants, four benchmarks, full efficiency profiling), and we then implement the catalogued extensions as a gated code layer and run the four that fit our GPU budget.

2 Experimental Setup

Model and variants. All experiments use Qwen2.5-7B-Instruct [13] at a pinned commit, run on a single RTX 4090 (24 GB). Every variant, including the bf16 baseline, runs on the same GPU, so the speed comparison is apples-to-apples with no cross-hardware caveat. Table 1 lists the five core variants. LLM.int8() and NF4 quantize the baseline weights at load time, so their disk footprint is the full bf16 checkpoint; GPTQ and AWQ are official pre-quantized 4-bit checkpoints served by the gptqmodel Marlin kernels.

Benchmarks and protocol. Quality is measured with the EleutherAI lm-evaluation-harness [6]: MMLU [7] (accuracy, aggregated with a weighting over its 57 subjects), GSM8K [2] (exact-match under flexible extraction, since the strict-match number is a formatting artefact for instruct models), HumanEval [1] (pass@1, with code execution), and IFEval [16] (prompt-level strict accuracy). Decoding is greedy and deterministic. MMLU and GSM8K use the chat template with 5-shot multi-turn prompting; HumanEval and IFEval are 0-shot. Efficiency is profiled separately (§3.2): the model is loaded once, then we measure disk size, load and generate peak VRAM, prefill and decode throughput at short, medium, and long prompts (128, 1024, and 4096 tokens), and the largest batch size before running out of memory.

Variant	Method	Precision	Source
bf16	half-precision baseline	W16	reference
bnb-int8	LLM.int8() [3]	W8 (load-time)	bitsandbytes
bnb-nf4	NF4 + double-quant [4]	W4 (load-time)	bitsandbytes
gptq-official	GPTQ [5]	W4 (g128)	official Qwen ckpt
awq-official	AWQ [10]	W4 (g128)	official Qwen ckpt

Table 1: The five core variants.

3 Results

3.1 Quality: capability sensitivity (RQ2)

Table 2 and Figure 1 report quality across the four benchmarks. The main finding is that quantization is remarkably benign for quality at 7B: every 4-bit variant stays within about 1.5 points of the bf16 baseline on MMLU and GSM8K. Sensitivity is uneven across capabilities, which answers RQ2.

Knowledge and math are close to lossless: MMLU and GSM8K barely move, and 8-bit is essentially unchanged. The MMLU drops are small but real: its standard error is tight (± 0.4), so the 4-bit variants sit measurably below bf16. On the other benchmarks the gaps are about the size of the standard error, so read those orderings as trends, not firm rankings. Coding shows the widest spread, with bnb-int8 losing about four points on HumanEval, the single largest drop; the 4-bit methods give up a point or two, though at HumanEval’s ± 3 -point standard error only the bnb-int8 gap approaches significance. Instruction following degrades only mildly, within about one standard error, and bnb-nf4 essentially ties the baseline. The 8-bit variant is the clearest illustration that more bits do not uniformly mean higher quality: it is near-lossless on knowledge and math yet gives the worst HumanEval score of any variant.

Variant	MMLU (acc)	GSM8K (flex. EM)	HumanEval (pass@1)	IFEval (strict)	Mean
bf16	73.6 ± 0.4	84.5 ± 1.0	84.1 ± 2.9	71.3 ± 1.9	78.4
bnb-int8	73.4 ± 0.4	84.4 ± 1.0	79.9 ± 3.1	69.3 ± 2.0	76.8
bnb-nf4	72.6 ± 0.4	83.6 ± 1.0	81.7 ± 3.0	71.0 ± 2.0	77.2
gptq-official	72.1 ± 0.4	83.2 ± 1.0	81.1 ± 3.1	70.2 ± 2.0	76.7
awq-official	72.2 ± 0.4	84.3 ± 1.0	81.1 ± 3.1	69.9 ± 2.0	76.9

Table 2: Quality (%), with standard error ($\pm$) shown per benchmark. Mean is a plain average across the four benchmarks (no standard error). Best per column in bold.

Quality by benchmark and quantization variant

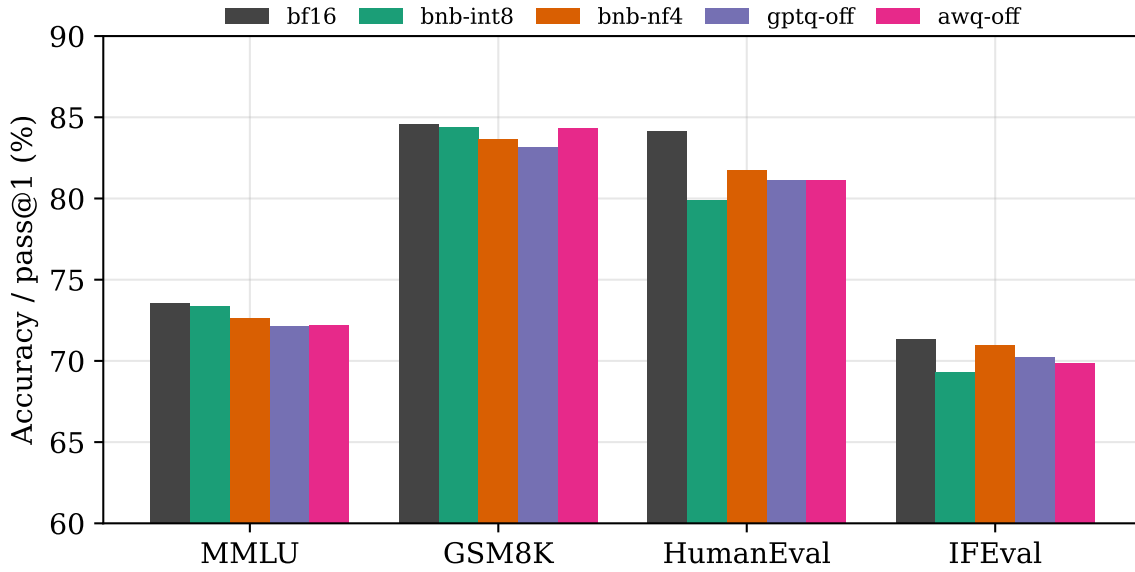


Figure 1: Quality by benchmark and variant. The 4-bit methods cluster tightly; HumanEval (coding) shows the widest spread, with bnb-int8 the most degraded.

3.2 Efficiency: memory vs. speed (RQ3)

The efficiency profile (Table 3, Figure 2) gives the clearest and most surprising findings, and answers RQ3 directly: quantization is primarily a memory technique, not a speed one.

LLM.int8() is the slowest variant. Its decode throughput collapses to about five times slower than bf16, because of the mixed-precision outlier decomposition, and it saves no disk since it stores the full bf16 checkpoint, making it the worst of both worlds on the speed and disk axes. NF4 is the memory-not-speed case: it cuts generate VRAM sharply and doubles the maximum batch, but still decodes below bf16. GPTQ and AWQ are the only variants that improve memory without paying much in speed. They cut disk by about 2.7 times, roughly halve generate VRAM, double the maximum batch, and hold decode to within about a quarter of baseline (28–29 vs. 37 tok/s), a modest slowdown next to the 2–5× of the other quantized variants. Prefill is unaffected across all variants, because quantization touches the memory-bound decode phase rather than the compute-bound prefill.

Variant	Disk (GiB)	Gen. VRAM (GiB)	Max batch	Decode (tok/s)	Prefill (tok/s)
bf16	14.19	14.96	16	37.0	9505
bnb-int8	14.19	9.22	16	7.2	9220
bnb-nf4	14.19	5.95	32	16.9	9196
gptq-official	5.19	5.94	32	28.5	9699
awq-official	5.19	5.96	32	27.9	9615

Table 3: Efficiency. Disk is the on-disk weight bytes; max batch is the largest power-of-two before OOM at 2048-token sequences; decode and prefill are at the 4096-token (long) bucket. Per-bucket throughput is in the appendix (Table 4).

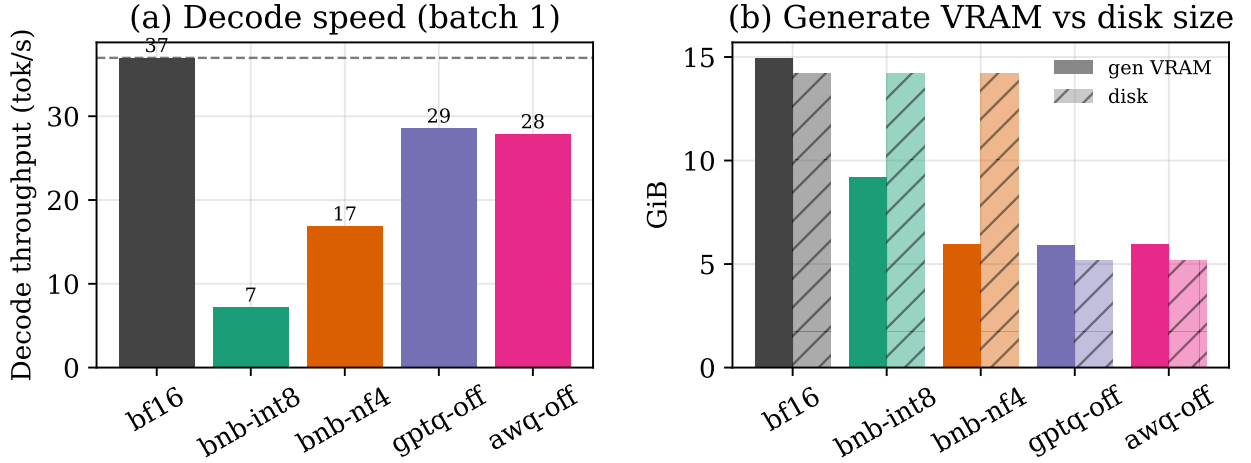


Figure 2: (a) Decode throughput; the dashed line marks bf16. bnb-int8 is about five times slower, and only GPTQ and AWQ stay near baseline. (b) Generate VRAM against on-disk size: the bitsandbytes variants shrink VRAM but not disk, while GPTQ and AWQ shrink both.

3.3 The trade-off and a recommendation (RQ1)

Figure 3 places all variants in the quality–speed–memory space, and the Pareto picture is clear. bf16 buys top speed at the highest memory; bnb-int8 is dominated, slower and more memory-hungry with no compensating quality gain; bnb-nf4 trades speed for memory; and GPTQ and AWQ sit at the sweet spot: the best quality–memory–speed balance, for a small quality cost. Under limited VRAM we recommend an official GPTQ or AWQ checkpoint. NF4 is the best fallback when no pre-quantized checkpoint exists, since it needs only the base weights, and LLM.int8() is hard to recommend on this hardware.

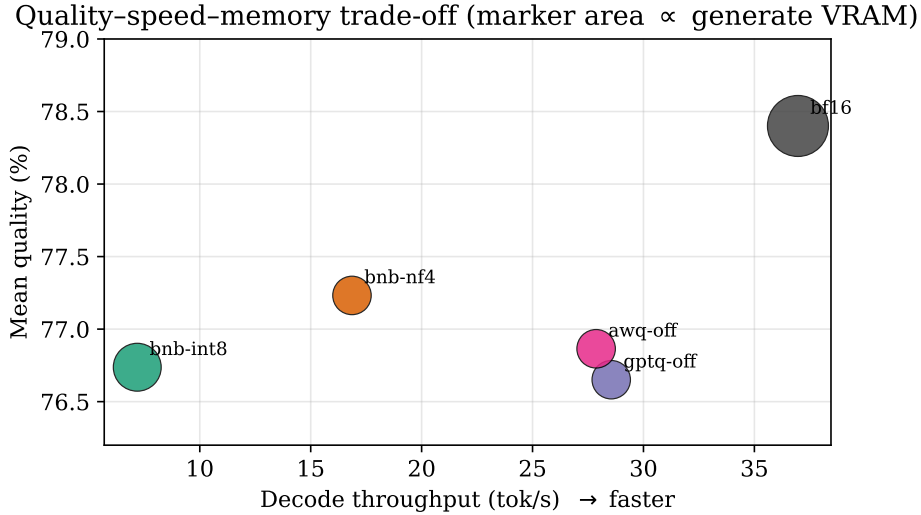


Figure 3: Quality–speed–memory trade-off. Up and to the right is better (higher mean quality, faster decode); smaller markers use less generate VRAM. GPTQ and AWQ occupy the high-speed, low-memory corner, and bnb-int8 is Pareto-dominated.

4 Extensions

Beyond the core study, we considered five optional extensions: JudgeBench, W8A8/SmoothQuant, a quantized KV cache, QLoRA recovery, and a second-model run. We ran the four that fit the GPU budget; adding Mistral-7B as a second model was skipped because it would have repeated the full compute campaign.

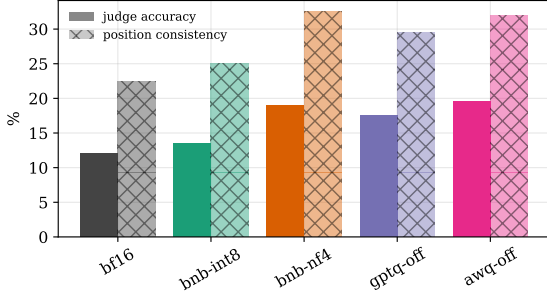
EXT-2: JudgeBench. We evaluate each variant as a pairwise judge on JudgeBench [14], a set of objective-correctness pairs, scoring agreement with the gold label under position-bias control, where both the A/B and B/A orders must agree. Figure 4a shows that quantization does not degrade judging: the 4-bit variants are, if anything, marginally better than bf16. The absolute numbers are low because Qwen2.5-7B is a weak, heavily position-biased judge on these hard pairs (only 22 to 33% of its verdicts are order-consistent), a property of the base model rather than of quantization.

EXT-3: W8A8 and SmoothQuant. We self-quantized a W8A8 checkpoint [15] (per-channel int8 weights with int8 activations) from the same C4 calibration set as our 4-bit self-quant recipes, using llm-compressor [12]. The checkpoint is produced and loads natively, but two things emerged on the HF runtime. First, compressed-tensors decompresses W8A8 to a bf16-resident tensor (15.2 GiB in VRAM), so it yields disk-only savings and no inference-memory benefit. Second, its `generate()` path is severely runtime-confounded, running at about 4.5 s per iteration, so a full quality sweep is impractical: the slowdown is itself the result. W8A8 is best understood as a vLLM or kernel-path technique, not an HF-generate one.

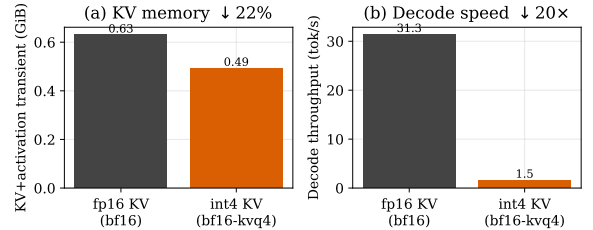
EXT-4: Quantized KV cache. Using the transformers v5 quantized cache (quanto, int4), we measured peak generation VRAM at a 3072-token context (Figure 4b). The int4 KV cache trims the KV-plus-activation transient by about 22%, but slows decode by roughly 20 times, because quanto dequantizes the cache on every step. This is a clear, quantitative confirmation of the runtime-confounded throughput caveat: a quantized KV cache trades a modest memory saving for a large ($\sim 20\times$) speed cost on the HF backend, and is only worthwhile with a fused kernel path such as KIVI [11] or vLLM.

EXT-5: QLoRA recovery (RQ4). We verified the full QLoRA [4, 8] recovery pipeline end-to-end: a LoRA adapter of rank 16, trained on the frozen NF4 base over general instruction data and guarded by a fail-fast check that the training corpus never overlaps the evaluation benchmarks, together with an adapter-aware load path. Training converged, and the adapted model loads with the adapter active and generates coherently. A full recovery measurement (enough training steps to move benchmark accuracy, followed by a re-evaluation) is a longer campaign we leave to future work; here we contribute the validated, contamination-safe machinery.

EXT-2 JudgeBench: pairwise judge accuracy & position consistency



(a) EXT-2 JudgeBench



(b) EXT-4 int4 KV cache

Figure 4: Extension results. (a) Judge accuracy and position-consistency per variant: quantization does not hurt judging. (b) int4 KV cache: about 22% less memory transient but roughly 20 times slower decode.

5 Conclusion

The main lesson is that quantization should be chosen for memory relief, not assumed speedup. On this hardware, the official GPTQ and AWQ checkpoints are the strongest default under limited VRAM because they preserve most quality while reducing both disk and generation memory. The bitsandbytes variants are useful in narrower cases: NF4 is a practical fallback when no pre-quantized checkpoint exists, while LLM.int8() is difficult to justify here because its runtime cost outweighs its accuracy advantage.

The capability results also argue against treating quantization error as uniform: knowledge and math were nearly stable, while coding was the most sensitive benchmark. The extensions reinforce the same deployment lesson. JudgeBench did not expose a quality failure from quantization, but W8A8 and int4 KV-cache experiments showed that backend support matters as much as bit-width. Our main limitations are the

single-model, single-seed evaluation and the HF-runtime confounds for self-quantization and KV-cache paths; multi-seed runs, the second model (Mistral-7B [9]), fused-kernel serving tests, and a full QLoRA recovery measurement (RQ4) are the natural next steps.

LLM Usage

In line with the course policy, we document our use of AI tools. The research design (the variant matrix, the quality–memory–speed framing, and the fairness and determinism protocol) is ours. We used an agentic coding assistant for supporting engineering: the torch-free orchestration layer and cloud-GPU run drivers, the evaluation and profiling harness scaffolding, the extension code layer, and the figure- and table-generation scripts. Every quantitative claim in this report was extracted programmatically from the committed result files and cross-checked, and agent-written code and prose were verified against the codebase and the logged metrics before inclusion.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL <https://arxiv.org/abs/2107.03374>.
- [2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [3] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://arxiv.org/abs/2208.07339>.
- [4] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.14314>.
- [5] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.17323>.
- [6] Leo Gao, Jonathan Tow, Baber Abbasi, et al. A framework for few-shot language model evaluation. EleutherAI lm-evaluation-harness, 2024. URL <https://github.com/EleutherAI/lm-evaluation-harness>.
- [7] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations (ICLR)*, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [8] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2106.09685>.
- [9] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023. URL <https://arxiv.org/abs/2310.06825>.
- [10] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Xingyu Dang, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*, 2024. URL <https://arxiv.org/abs/2306.00978>.
- [11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *International Conference on Machine Learning (ICML)*, 2024. URL <https://arxiv.org/abs/2402.02750>.

- [12] Neural Magic / Red Hat. llm-compressor and compressed-tensors. GitHub repository, 2024. URL <https://github.com/vllm-project/llm-compressor>.
- [13] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [14] Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y. Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. JudgeBench: A benchmark for evaluating LLM-based judges. *arXiv preprint arXiv:2410.12784*, 2024. URL <https://arxiv.org/abs/2410.12784>.
- [15] Guangxuan Xiao, Ji Lin, Mickael Seznec, Hao Wu, Julien Demouth, and Song Han. SmoothQuant: Accurate and efficient post-training quantization for large language models. In *International Conference on Machine Learning (ICML)*, 2023. URL <https://arxiv.org/abs/2211.10438>.
- [16] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023. URL <https://arxiv.org/abs/2311.07911>.

A Reproducibility and Selected Efficiency Details

Setup. Single RTX 4090 (24 GB); torch 2.12.1+cu126, transformers 5.10.1, lm-eval 0.4.3 (0.4.12 for HumanEval, whose code-execution harness requires the newer release), bitsandbytes 0.49.2, gptqmodel 7.1.0, compressed-tensors 0.17.1. Greedy decoding; seeds fixed (random, numpy, torch, few-shot). Quality and efficiency are profiled in separate phases, and each variant is loaded once. All result JSONs and the extension and figure code are in the project repository; the figures and tables in this report are generated directly from those files.

Variant	Bucket	Prompt (tok)	Prefill (tok/s)	Decode (tok/s)	TTFT (s)	E2E lat. (s)
bf16	short/long	128/4096	4141/9505	37.0/37.0	0.03/0.43	6.93/7.33
bnb-int8	short/long	128/4096	−/9220	7.2/7.2	—	—
bnb-nf4	short/long	128/4096	−/9196	16.9/16.9	—	—
gptq-official	short/long	128/4096	−/9699	28.3/28.5	—	—
awq-official	short/long	128/4096	−/9615	28.0/27.9	—	—

Table 4: Per-bucket throughput (selected fields; short/long = 128/4096-token prompts). Decode is near-constant across prompt length, as expected for a memory-bandwidth-bound phase. Full TTFT and latency per bucket are in the result JSONs.

Extension artifacts. Adding the counts omitted above: EXT-2 covered n=200 JudgeBench pairs per variant, and EXT-3’s self-quantized W8A8 checkpoint is 8.7 GiB on disk.